KTH

DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING
SECOND CYCLE, 30 CREDITS

Stockholm, Sweden 2026

Joint Radar Emitter and Mode Clustering Using a Transformer Encoder Architecture

Markus Swegmark

Supervisor: TBD
Examiner: TBD
Host: TBD

KTH Royal Institute of Technology
School of Electrical Engineering and Computer Science
Master's Programme in Machine Learning

TRITA-EECS-EX-XXXX:XXX

Abstract

This thesis investigates joint clustering of radar emitters and their operating modes from intercepted pulse descriptor word (PDW) streams using a transformer encoder architecture.

Keywords: transformer, deep clustering, electronic intelligence, radar emitter identification, mode recognition, self-supervised learning.

Sammanfattning

Detta examensarbete undersöker gemensam klustring av radarsändare och deras operativa moder från avlyssnade pulsbeskrivande ord (PDW) med hjälp av en transformerbaserad encoderarkitektur.

Nyckelord: transformer, djup klustring, signalspaning, radarsändaridentifiering, modigenkänning, självövervakad inlärning.

Acknowledgments

I would like to thank my supervisor and examiner at KTH, and the host organization for their support throughout this thesis project.

Stockholm, May 2026
Markus Swegmark

Contents

Abstract i

Sammanfattning ii

Acknowledgments iii

List of Acronyms viii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Questions . . . 2
- 1.4 Objectives and Scope . . . 2
- 1.5 Contributions . . . 2
- 1.6 Ethical and Societal Considerations . . . 2
- 1.7 Thesis Outline . . . 2

2 Background 4
- 2.1 Radar Systems and Passive Reception . . . 4
- 2.2 Pulse Descriptor Words . . . 5
- 2.3 Radar Emitters and Operating Modes . . . 5
- 2.4 Signal Processing Pipeline for Passive ELINT . . . 6
- 2.5 Machine Learning Fundamentals . . . 6
- 2.6 Clustering Methods . . . 8
- 2.7 Deep Representation Learning . . . 8
- 2.8 The Transformer Encoder . . . 9

3 Related Work 11
- 3.1 Classical Radar Emitter Identification . . . 11
- 3.2 Deep Learning for Radar Signal Analysis . . . 11
- 3.3 Deep Clustering . . . 11
- 3.4 Transformers for Time Series and Sets . . . 11
- 3.5 Joint and Multi-Task Clustering . . . 11

4 Method 12
- 4.1 Problem Formulation . . . 12
- 4.2 Data Representation and Preprocessing . . . 14
- 4.3 Transformer Encoder Architecture . . . 15
- 4.4 Loss Function and Clustering Objective . . . 18

4.5 Training Procedure 21
4.6 Evaluation Metrics 22
4.7 Baseline Methods 24

5 Experiments and Results **25**
5.1 Experimental Setup 25
5.2 Datasets 25
5.3 Hyperparameters and Training Details 26
5.4 Quantitative Results 27
5.5 Ablation Studies 27
5.6 Qualitative Analysis 27

6 Discussion **28**
6.1 Interpretation of Results 28
6.2 Comparison with Related Work 28
6.3 Limitations 28
6.4 Implications 28

7 Conclusion **29**
7.1 Summary 29
7.2 Answers to the Research Questions 29
7.3 Future Work 29

References **30**

A Additional Results **32**

B Implementation Details **33**

C Notation Reference **34**

List of Figures

4.1 Illustration of a PDW stream $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ in time order. Each cell combines emitter color (fill) and operating-mode pattern; the in-cell vector index is typeset in black. The emitter legend uses color-only swatches with matching colored text; the mode legend uses grayscale pattern swatches with black text. Emitter clustering seeks a partition of indices into subsets $\mathcal{E}_k$ (constant e_n on each subset); mode clustering seeks a second partition into subsets $\mathcal{M}_\ell$ (constant m_n), while allowing mode hops within a fixed emitter. 13

4.2 Architecture of the encoder $f_{\boldsymbol{\theta}}$. A sequence of PDWs is first lifted to $d_{\text{model}} = 256$ by an input embedding, processed by a shared trunk of 6 transformer-encoder layers, and then split into two parallel branches with their own $256 \to 128$ projection and 2 encoder layers. Branch outputs are mapped by linear projection heads to the emitter embedding space $\mathbb{R}^8$ and the mode embedding space $\mathbb{R}^{16}$. 16

List of Tables

5.1 Synthetic scenario corpus used for training and evaluation. Dashes mark quantities still being reconciled with the export manifest. 26
5.2 Primary optimisation hyperparameters for the proposed model. 26

List of Acronyms

ACC clustering accuracy

AMI adjusted mutual information

AOA angle of arrival

ARI adjusted Rand index

BERT Bidirectional Encoder Representations from Transformers

DEC deep embedded clustering

DL deep learning

ELINT electronic intelligence

ESM electronic support measures

EW electronic warfare

FFN feed-forward network

HDBSCAN Hierarchical density-based spatial clustering of applications with noise

MHA multi-head attention

ML machine learning

MLP multilayer perceptron

NMI normalized mutual information

PDW pulse descriptor word

PRF pulse repetition frequency

PRI pulse repetition interval

RADAR radio detection and ranging

RF radio frequency

SNR signal-to-noise ratio

TOA time of arrival

V-measure harmonic mean of homogeneity and completeness

CHAPTER 1

Introduction

1.1 Background and Motivation

Recent large-scale conflicts illustrate how contested the electromagnetic spectrum has become. Commanders rely on timely, passive sensing to build situational awareness, protect platforms, and coordinate fires without revealing their own emissions. Within EW, which aims to exploit, protect, and maneuver in the spectrum, ESM systems listen passively and supply the measurements that feed higher-level ELINT tasks such as detecting radars, grouping pulses by emitter, and inferring behavior [1, 2].

Modern radars are increasingly agile: they vary carrier frequency, pulse width, amplitude, and PRI across bursts to improve performance and survivability. That agility blurs simple fingerprints and stresses classical rule-based parsers tuned to stable pulse trains. The same physical emitter also switches *modes*—recurrent control policies that map an operational objective (for example, volume search versus track) to a characteristic regime in the pulse train. Mode awareness is therefore operationally informative: it constrains how the waveform may evolve, signals intent at a coarse level, and can help disambiguate emitters when type-level signatures overlap.

At the same time, ML and DL have become the default toolkit for sequence modeling and representation learning in other domains. Radar pulse analysis naturally presents as a stream of low-level measurements—often summarized as PDWs—where long-range dependence and mixture structure (multiple emitters superposed in time) favor learned encoders over purely hand-crafted feature pipelines, even when labels exist only for development or evaluation data.

This thesis studies whether a transformer-style encoder, trained with objectives suited to clustering, can produce embeddings in which both distinct emitters and their operating modes emerge as separable structure. The remainder of the introduction states the problem precisely, lists the research questions, and outlines scope, contributions, and ethics.

1.2 Problem Statement

Passive ESM produces a time-ordered stream of PDWs in which pulses from multiple emitters are interleaved and each emitter may change operating mode over time (Section 1.1). Dense per-pulse supervision for emitter identity and mode is rarely available outside curated datasets, yet operators still require both emitter grouping and coarse mode structure from the same measurement stream. Classical pipelines therefore often

chain deinterleaving with separate mode heuristics or supervised classifiers trained on narrow taxonomies; under agile waveforms and overlapping trains, that separation scales poorly and does not guarantee a single representation whose geometry exposes emitter- and mode-level structure simultaneously for clustering-based analysis. The problem addressed in this thesis is to learn, from such an unlabeled pulse stream, an embedding in which unsupervised clustering recovers distinct emitters and, to the extent supported by the features, operating modes, while remaining robust to mixture effects, noise, and imperfect preprocessing. Labels, when present, serve evaluation and objective specification in controlled settings rather than defining the operational learning target.

1.3 Research Questions

The thesis addresses the following research questions:

RQ1 Under practical sequence-length limits of a transformer encoder, can the model support effective deinterleaving and recover operating modes via clustering?

RQ2 Does a joint objective that couples deinterleaving with mode-aware clustering yield better clustering performance or representations than a comparable single-objective transformer baseline focused on deinterleaving alone?

1.4 Objectives and Scope

Placeholder paragraph.

1.5 Contributions

The main contributions of this thesis are:

- Placeholder contribution.
- Placeholder contribution.

1.6 Ethical and Societal Considerations

Placeholder paragraph.

1.7 Thesis Outline

The remainder of this thesis is organized as follows. Chapter 2 introduces the necessary background on radar signal processing and deep learning. Chapter 3 surveys related work. Chapter 4 describes the proposed method. Chapter 5 presents experiments and

results. Chapter 6 discusses the findings. Chapter 7 concludes and outlines future work.

CHAPTER 2

Background

2.1 Radar Systems and Passive Reception

Electromagnetic waves carry energy from a transmitter to one or more receivers. The same physical mechanism supports both communications and remote sensing: a receiver can infer information from the field strength, timing, and structure of an arriving wave. RADAR exploits this idea for echolocation at radio frequencies by comparing transmitted energy to echoes scattered from objects of interest [2].

In a typical pulsed system, a collocated or nearby transmitter and receiver emit short bursts and listen during the intervals between pulses. Echo delay maps to range under the speed-of-light propagation model, while angular information comes from antenna directivity or multi-channel processing. Successive pulses are repeated with a nominal PRI (equivalently, a PRF), which sets the unambiguous range window and the illumination duty cycle. On receive, matched filtering and integration improve the effective SNR before detection and parameter estimation [2].

For a monostatic configuration with common transmit and receive gain G, wavelength λ, target radar cross section σ, and slant range R, the received power obeys the familiar scaling

$$P_r = \frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 R^4}, \tag{2.1}$$

where P_t is the peak transmit power during the pulse [2]. Equation (2.1) shows why range and antenna gain dominate link budgets: P_r falls approximately as R^{-4}, so small changes in geometry or losses can produce large swings in echo strength. In practice, designers also account for atmospheric attenuation, system losses, and processing gains; the simplified form is nevertheless the standard reference for how "what the radar transmits" couples to "what comes back."

Passive ESM and ELINT systems observe the same emissions from the sidelobe or mainlobe of a third-party radar, but without control of P_t, G, or the waveform timing on transmit. The analyst therefore works from one-way propagation, often with incomplete geometry, and must infer emitter behavior from measurable pulse parameters rather than from a known matched filter reference. Those per-pulse measurements and their downstream representations are the usual interface to learning-based methods; Section 2.2 summarizes the corresponding feature view.

2.2 Pulse Descriptor Words

Placeholder paragraph.

2.3 Radar Emitters and Operating Modes

In passive ESM and ELINT, a *radar emitter* denotes a physical transmitting source—typically a platform-mounted set—whose emissions can be tied to a coherent train of pulses in time and frequency [1]. An *operating mode*, by contrast, is a parameterized configuration of the transmitted waveform and scheduling policy: nominal carrier frequency or hopping law, pulse width, repetition structure, beam scan program, and any agility rules that govern how those quantities change from pulse to pulse [1, 2]. Modes are therefore properties of the emission process at a given time, whereas emitters are stable identities of hardware and platform; the same hardware cycles through modes as the mission evolves.

Illustrative mode families include wide-area search, acquisition and track, fire-control illumination, weather sensing, and terminal guidance support [2]. Each family implies characteristic constraints on PRI/PRF, dwell time, frequency agility, and antenna motion: search modes favor broad coverage and may use staggered or agile repetition to balance range and velocity ambiguity, while dedicated tracking or illuminator modes often concentrate energy in fewer directions with higher update rates on a target of interest. Navigation radars on ships and aircraft emphasize short-range obstacle and surface mapping; their emissions tend to repeat with comparatively regular timing tied to a fixed sector or rotating beam pattern, whereas long-range surveillance and fire-control systems interleave search and track waveforms and may switch RF bands or burst structures when countering interference [1, 2]. The resulting diversity matters for downstream analysis because operators must interpret not only "which radar," but also "what it is doing now," and because the same measured PDW parameters can arise from different combinations of hardware and scheduling (Section 2.2).

The relationship between emitters and modes is many-to-many. A single emitter implements many modes over a mission timeline, so a time-ordered pulse stream is a mixture of mode segments whose boundaries need not align with changes of physical source. Conversely, different emitters can realize the same functional mode with different parameter sets—for example, two fire-control radars may both run a track waveform but with distinct nominal PRFs and frequency plans. That structure motivates joint reasoning about physical identity and instantaneous configuration: clustering only over pulses loses scheduling context, whereas treating modes without emitter linkage obscures which trains belong together after deinterleaving errors or overlap [1]. The remainder of this chapter situates passive reception in the EW/ESM processing chain (Section 2.4) and then supplies the learning background needed to formalize such joint structure in the method.

2.4 Signal Processing Pipeline for Passive ELINT

In EW and ESM applications, measured radar energy passes through a staged *signal processing pipeline* before it reaches higher-level ELINT reasoning. The exact implementation varies with platform, band, and mission software, but the logical chain is broadly stable across textbooks and system descriptions.

At the *front end*, antennas and RF conditioning capture emissions over one or more instantaneous bandwidths, often after wideband digitization or channelized sub-band processing. A *detection* stage then marks pulse-like events against a noise and clutter background, producing a sparse list of candidate pulse times (and sometimes coarse frequency hints).

Given detections, *parameter estimation* attaches per-pulse measurements such as carrier frequency, pulse width, time of arrival, direction of arrival when available, and amplitude or signal-to-noise proxies. These estimates are noisy, can be missing for weak emitters, and may include false alarms that are not associated with any physical radar.

Sorting and *deinterleaving* group pulses into coherent pulse trains by exploiting periodic structure in inter-pulse timing and consistent parameter tracks across time. The output of this stage is commonly packaged as a stream of PDWs (or closely related pulse descriptor records), which form the usual interface to downstream tasks such as emitter identification and mode analysis (Section 2.2).

In classical ELINT architectures, specifications therefore treat train formation as an early, largely discrete step: deinterleaving is completed first so that downstream logic receives one dominant hypothesis per physical emission path, or a short ranked list, rather than a time-interleaved mixture [1]. *Emitter reports* are assembled only after that gate, summarizing each train with aggregated PDW statistics, tentative equipment identities from lookup tables, and temporal extent. Mode classification is then commonly framed as matching the train against a *mode library* of nominal RF bands, PRI/PRF families, stagger laws, and scan programs, so that declared modes remain anchored to catalogued waveform templates. *Positional estimation*—for example AOA intersections from direction-finding networks, time-difference-of-arrival fixes along baselines, or fused tracks across collectors—consumes the same deinterleaved associations, which isolates geometry and timing errors from the sorting stage.

Later stages may maintain libraries, threat lists, and operator displays, but for the learning problem in this thesis the critical contract is that the model consumes the deinterleaved PDW stream produced here.

2.5 Machine Learning Fundamentals

ML problems are often categorized by the supervision available at training time. In the supervised setting, a learner fits a mapping from inputs to labels by minimizing an empirical risk over a finite set of annotated examples. Unsupervised learning instead

seeks structure in the input distribution itself, for example through density estimation, dimensionality reduction, or clustering, without externally supplied targets. Self-supervised learning occupies an intermediate regime: targets are derived automatically from the data through a designed pretext (for example predicting masked parts of an input or contrasting augmented views), which supplies learning signal without manual annotation. Across these regimes, a common formalism is empirical risk minimization, in which one minimizes an average training loss with respect to model parameters and relies on optimization, regularization, and held-out evaluation to control generalization.

Much of modern DL practice is best understood as *representation learning*: a parametric encoder maps high-dimensional observations into a lower-dimensional vector space whose geometry is shaped by the training objective. Downstream tasks—classification, clustering, retrieval, or sequence prediction—then operate on these *embeddings* rather than on raw sensor coordinates. The thesis later instantiates this pattern for PDW streams by training an encoder whose outputs are intended to expose emitter and mode structure (Chapter 4).

LeCun [3] argues for organizing this picture around *joint embeddings*. In joint embedding predictive architectures, two related observations x and y (for example two temporal crops of the same scene) are mapped to representations s_x and s_y by encoders, and a predictor proposes $\tilde{s}_y$ from s_x (possibly with latent variables that capture multi-valued futures). Compatibility is scored by an energy $D(s_y, \tilde{s}_y)$ that measures prediction error in representation space rather than in pixel space or other raw domains. As LeCun [3] puts it,

> "The energy is simply the prediction error in representation space."

The same source emphasizes that the main advantage of this design is that prediction happens after irrelevant variability has been discarded by the encoders, so that the learning problem is not to reconstruct every detail of y, but to align abstract embeddings that already summarize what must stay predictable.

When D is instantiated as a squared Euclidean distance between embeddings, the landscape of D with respect to s_y is piecewise quadratic under standard predictor parameterizations. LeCun [3] uses this viewpoint to explain why, in discrete-latent variants, low-energy regions can be visualized as competing wells in embedding space:

> "In representation space, the energy will be the minimum of K quadratic energy wells."

Colloquially, one can read this as the claim that *once* suitable embeddings and predictors have been identified, much of the remaining compatibility training reduces to a family of quadratic matching problems in representation coordinates; the scientific effort then concentrates on representation geometry, collapse prevention, and task-relevant invariances rather than on inventing elaborate scalar losses on raw inputs. The clustering objectives used in Chapter 4 should be interpreted against that backdrop: they reshape embedding space, while the parametric form of the pointwise loss can remain simple.

2.6 Clustering Methods

Placeholder paragraph.

2.7 Deep Representation Learning

Many DL pipelines replace hand-crafted features with a parametric map that is trained end-to-end from data. The central object is an *encoder* $f_{\boldsymbol{\theta}}\colon \mathcal{X} \to \mathbb{R}^d$, implemented in practice as a stack of nonlinear layers (for example, convolutions over a grid, recurrence over time, or attention over a sequence) followed by a pooling or readout head that yields a fixed-dimensional embedding $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x})$. The design question is which training signal makes $\boldsymbol{z}$ useful for the downstream task when labels are scarce or absent.

In supervised learning, $f_{\boldsymbol{\theta}}$ is trained together with a classifier so that $\boldsymbol{z}$ separates classes under a cross-entropy or margin loss. When cluster structure is the target but assignments are unknown, the same encoder can instead be trained with objectives that encourage smoothness, invariance to nuisance transformations, or agreement with a clustering hypothesis in embedding space. *Self-supervised* learning occupies an intermediate regime: the model predicts surrogate targets constructed from the input itself (for example, masked tokens, relative positions, or transformed views), so that $f_{\boldsymbol{\theta}}$ learns semantics without human labels [4].

Contrastive objectives implement instance discrimination by treating two augmentations of the same example as a positive pair and other examples in a minibatch (or a memory bank) as negatives. Let $\boldsymbol{z}_i$ and $\boldsymbol{z}_i^{+}$ denote normalized embeddings of two views of sample i, let $\mathrm{sim}(\cdot,\cdot)$ denote cosine similarity, and let $\tau > 0$ be a temperature. A standard *InfoNCE*-style loss takes the form

$$\mathcal{L}_{\mathrm{NCE}}(i) = -\log \frac{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^{+})/\tau\Big)}{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^{+})/\tau\Big) + \sum_{j \neq i} \exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_j^{+})/\tau\Big)}, \tag{2.2}$$

which lower-bounds mutual information between representations of different views under suitable generative assumptions [5]. Large-scale implementations such as SimCLR combine strong augmentations with wide embeddings and many negatives drawn from the same minibatch [4]. Equation (2.2) should be read as a template: the positive set can be enlarged, negatives can be drawn from a queue, and the similarity can be implemented with a learned projection head without changing the underlying contrastive principle.

Pretext tasks offer an alternative route: the network solves an auxiliary problem (predicting rotations, solving jigsaw permutations, inpainting masked regions) whose solution is incidental, but the representation $\boldsymbol{z}$ becomes predictive of semantically stable factors. Such objectives can be combined with clustering when the end goal is partition discovery rather than linear probe accuracy on a fixed label set.

DEC couples an encoder with a soft assignment to a small set of cluster centroids in embedding space and trains the model by minimizing a Kullback–Leibler divergence between the soft assignments and an auxiliary target distribution that sharpens confident predictions [6]. SwAV replaces pairwise instance contrast with *prototype* classification: views are mapped to cluster codes, and the loss enforces prediction consistency across augmentations while preventing collapse through swapped assignments [7]. Both families treat the encoder as the shared backbone through which raw inputs are mapped to a geometry where classical clusterers or learned assignment heads operate.

Sequence encoders based on self-attention, including the transformer blocks reviewed in Section 2.8, instantiate $f_{\boldsymbol{\theta}}$ when $\boldsymbol{x}$ is an ordered set of tokens (for example, a window of PDWs). The Method chapter builds on this stack: a contrastive or clustering-friendly loss shapes the embedding space, while the attention-based encoder models temporal context within each window.

2.8 The Transformer Encoder

The transformer architecture replaces recurrence with a stack of layers built around *self-attention*, so that every position in a sequence can directly attend to every other position in parallel [8]. Encoder-only stacks, as in BERT [9], map an input sequence to contextualised token representations of the same length and are a natural choice when the downstream task is representation learning or clustering over tokens rather than autoregressive generation.

Scaled dot-product attention. Attention maps three matrices of *queries* $\boldsymbol{Q}$, *keys* $\boldsymbol{K}$, and *values* $\boldsymbol{V}$ to an output matrix of the same leading dimension as $\boldsymbol{Q}$. Each row of $\boldsymbol{Q}$ scores all rows of $\boldsymbol{K}$ through inner products; the scores are scaled, normalised with a row-wise softmax, and used to form a convex combination of the rows of $\boldsymbol{V}$. With d_k the dimension of each query and key vector (column width of $\boldsymbol{Q}$ and $\boldsymbol{K}$ after the usual transpose convention), the mapping is

$$\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}. \tag{2.3}$$

The scaling by $\sqrt{d_k}$ keeps the inner products from growing too large in magnitude as d_k increases, which would otherwise drive the softmax into near one-hot rows and yield vanishing gradients [8]. Equation (2.3) is reused later as the core attention primitive inside the encoder.

Self-attention and MHA. In *self-attention*, $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are all linear projections of the same input sequence layout, so a token's representation is updated by aggregating information from the full sequence. MHA runs h attention mechanisms in parallel on different learned subspaces, concatenates the head outputs, and applies one more linear

map to restore the model width [8]. The heads specialise to different relational patterns while sharing the same residual and normalisation wrapper as a single conceptual operator.

Positional information. Self-attention is permutation-equivariant in its inputs: reordering the rows of $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ in the same way reorders the output rows accordingly, but does not otherwise encode order. Transformer encoders therefore add a positional signal to token embeddings before the first layer, for example fixed sinusoidal features or learned position embeddings added element-wise to each token vector [8, 9]. When the raw inputs already carry absolute or relative timing, as is common for irregularly sampled sequences, practitioners sometimes omit a separate positional encoding and rely on those measurements instead; the method chapter returns to this choice for PDW windows.

Encoder layer. A standard transformer *encoder layer* alternates two sub-layers: MHA followed by a position-wise FFN applied independently at every position. Each sub-layer uses a residual connection and layer normalisation [8]. The FFN is typically a two-layer MLP with a wide inner dimension and a nonlinear activation, acting as the per-token nonlinearity while MHA handles cross-token mixing. Stacking N such layers yields a deep encoder that maps an input sequence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_L)$ to contextualised representations $(\boldsymbol{h}_1^{(N)}, \ldots, \boldsymbol{h}_L^{(N)})$, which downstream heads can pool or read out for sequence-level or token-level predictions.

CHAPTER 3

Related Work

3.1 Classical Radar Emitter Identification

Placeholder paragraph.

3.2 Deep Learning for Radar Signal Analysis

Placeholder paragraph.

3.3 Deep Clustering

Placeholder paragraph.

3.4 Transformers for Time Series and Sets

Placeholder paragraph.

3.5 Joint and Multi-Task Clustering

Placeholder paragraph.

CHAPTER 4

Method

4.1 Problem Formulation

This chapter studies unsupervised structure recovery from a time-ordered stream of PDWs produced by an ESM or deinterleaving front end, as outlined in Section 2.4. The goal is twofold: associate each pulse with a physical *emitter instance* and, within that instance, with an *operating mode*, without prior knowledge of how many emitters or modes appear in the data. Both quantities are latent because labels are scarce in ELINT settings and because novel emitters and modes must be accommodated.

Observed input

Let a sequence of N pulses be represented by feature vectors

$$\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N), \qquad \boldsymbol{x}_n \in \mathbb{R}^d, \tag{4.1}$$

where each $\boldsymbol{x}_n$ encodes the measured pulse parameters associated with one PDW (carrier frequency, pulse width, timing, spatial cues, and any auxiliary fields). The sequence may contain irregular sampling, missing pulses, and spurious detections inherited from upstream processing; robustness to such effects is a design constraint rather than part of the formal specification.

Latent emitter and mode assignments

For each index $n \in \{1, \ldots, N\}$, let e_n denote the emitter identity responsible for pulse n and let m_n denote the operating mode within that emitter. These variables take values in finite label sets whose cardinalities

$$K_{\text{em}} = |\{e_1, \ldots, e_N\}|, \qquad K_{\text{md}} = |\{m_1, \ldots, m_N\}| \tag{4.2}$$

are *unknown a priori* and need not match the number of physical platforms in the environment (for example, co-located antennas may map to a single inferred emitter index, or a single platform may split across indices when observations are ambiguous). The pair (e_n, m_n) is not observed at training time; the learning problem is to infer representations from which both clusterings can be recovered reliably.

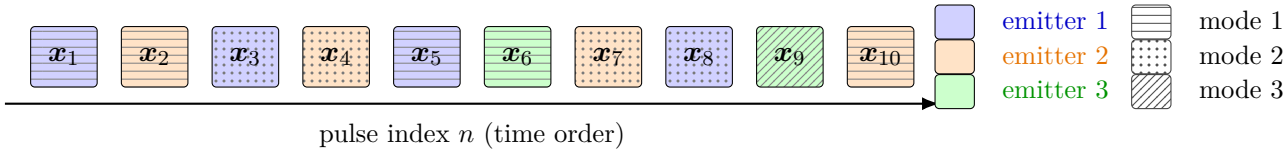

Figure 4.1. Illustration of a PDW stream $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ in time order. Each cell combines emitter color (fill) and operating-mode pattern; the in-cell vector index is typeset in black. The emitter legend uses color-only swatches with matching colored text; the mode legend uses grayscale pattern swatches with black text. Emitter clustering seeks a partition of indices into subsets $\mathcal{E}_k$ (constant e_n on each subset); mode clustering seeks a second partition into subsets $\mathcal{M}_\ell$ (constant m_n), while allowing mode hops within a fixed emitter.

Induced partitions, PDW streams, and mode hopping

Figure 4.1 sketches a short PDW stream in chronological order. Fill color encodes the latent emitter index e_n; pulses that share a color belong to the same emitter-specific subset of indices even though they need not be contiguous in time (interleaved emissions from several RADAR platforms are typical). Each stream cell uses that emitter fill *and* a grayscale pattern for operating mode m_n; the vector label $\boldsymbol{x}_i$ inside each cell is drawn in black for legibility. The right-hand legends separate the two cues: emitter entries are color-only (no pattern), and mode entries are monochrome pattern tiles with black text. The formal partitions and mode hopping are stated mathematically in the text below.

Emitter clustering is the recovery of the partition of $\{1, \ldots, N\}$ into emitter-coherent index sets

$$\mathcal{E}_k = \{\, n \in \{1, \ldots, N\} : e_n = k \,\}, \qquad k \in \{1, \ldots, K_{\mathrm{em}}\}, \tag{4.3}$$

so that $\boldsymbol{x}_n$ and $\boldsymbol{x}_{n'}$ are grouped together if and only if $e_n = e_{n'}$. Equivalently, each $\mathcal{E}_k$ is the preimage of label k under the map $n \mapsto e_n$, and $\{\mathcal{E}_1, \ldots, \mathcal{E}_{K_{\mathrm{em}}}\}$ is pairwise disjoint with union $\{1, \ldots, N\}$.

Mode clustering targets an analogous partition into mode-coherent subsets,

$$\mathcal{M}_\ell = \{\, n \in \{1, \ldots, N\} : m_n = \ell \,\}, \qquad \ell \in \{1, \ldots, K_{\mathrm{md}}\}, \tag{4.4}$$

which groups pulses that share the same operating mode regardless of which physical emitter produced them. The two partitions are coupled because (e_n, m_n) is generated jointly: restricting to a fixed emitter k, the induced set of mode labels

$$\mathcal{L}_k = \{\, m_n : n \in \mathcal{E}_k \,\} \tag{4.5}$$

has cardinality $L_k = |\mathcal{L}_k|$ bounded by the number of distinct waveform programmes that emitter k exercises. In many RADAR systems, mode hopping switches among a small, essentially finite catalogue of modes, so L_k is modest—often on the order of single-digit integers—even though the temporal pattern $(m_n)_{n \in \mathcal{E}_k}$ is not constant. The learning problem is to infer partitions that match $\{\mathcal{E}_k\}$ and $\{\mathcal{M}_\ell\}$ up to independent relabellings (permutations of emitter symbols k and mode symbols ℓ) from $\boldsymbol{X}$ alone,

using the dual embeddings in Equation (4.6).

Desired outputs: dual embeddings and discrete labels

The parametric model considered in this thesis maps each input sequence to two parallel trajectories in Euclidean embedding spaces,

$$\boldsymbol{Z}^{\mathrm{em}} = (\boldsymbol{z}_1^{\mathrm{em}}, \ldots, \boldsymbol{z}_N^{\mathrm{em}}), \quad \boldsymbol{z}_n^{\mathrm{em}} \in \mathbb{R}^p, \qquad \boldsymbol{Z}^{\mathrm{md}} = (\boldsymbol{z}_1^{\mathrm{md}}, \ldots, \boldsymbol{z}_N^{\mathrm{md}}), \quad \boldsymbol{z}_n^{\mathrm{md}} \in \mathbb{R}^q, \tag{4.6}$$

with dimensions p and q fixed by the architecture. Discrete emitter labels $\hat{e}_n$ and mode labels $\hat{m}_n$ are obtained *post hoc* by clustering $\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^N$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}_{n=1}^N$, or by reading off prototype assignments when the training objective includes cluster parameters. Thus the immediate trainable targets are the continuous vectors in Equation (4.6); hard partitions are induced indirectly.

The two trajectories serve different roles. Emitter-oriented vectors $\boldsymbol{z}_n^{\mathrm{em}}$ are primarily a vehicle for estimating consistent emitter membership; their main deliverable is a per-pulse emitter index suitable for downstream ELINT workflows such as emitter counting and localisation. Mode-oriented vectors $\boldsymbol{z}_n^{\mathrm{md}}$ additionally aim to summarise mode-dependent waveform behaviour so that operating modes can be inspected, compared, or matched against a library of known modes once such references become available.

Architecturally, both streams are produced from a shared encoder $f_{\boldsymbol{\theta}}$ with task-specific heads (see Section 4.3); in deployment, emitter embeddings may be less critical to retain after discrete $\hat{e}_n$ have been fixed, whereas mode embeddings (or intermediate encoder activations feeding the mode head) are natural inputs to specialised mode classifiers.

Learning objective

The optimisation problem couples representation learning with clustering-friendly losses so that the geometry of $\boldsymbol{Z}^{\mathrm{em}}$ and $\boldsymbol{Z}^{\mathrm{md}}$ aligns with emitter and mode structure, respectively. The concrete loss terms and training protocol are given in Sections 4.4 and 4.5.

4.2 Data Representation and Preprocessing

ELINT data are difficult to obtain at scale, and operational recordings are often sensitive. This work therefore relies on synthetic pulse trains from a proprietary scenario simulator (Saab), which yields controlled scenarios and full supervision of emitter identity and operating mode for evaluation, while the learning objectives remain unsupervised with respect to those labels.

Each scenario is a time-ordered sequence of PDWs together with emitter and mode annotations. Scenarios vary in the number of emitters and in sequence length

so that training covers diverse traffic densities and temporal horizons; exact counts, train/validation/test splits, and sampling ranges are reported in Chapter 5.

Windowing and context length

The encoder operates on fixed-length windows because self-attention cost scales quadratically with sequence length. Let L denote the window length (in pulses) fed to the model; experiments use $L = 1024$. Each scenario is partitioned into contiguous windows of length L. If the remainder after the last full window is shorter than L, it is dropped in the present pipeline; padding with an attention mask would retain the tail without changing the architecture and is left as a straightforward extension.

Training windows are extracted with a stride of $L/2$, so consecutive windows overlap by half their length. This increases the number of training pulses seen per scenario and reduces sensitivity to the particular alignment of window boundaries relative to mode transitions. Validation and test windows use stride L (no overlap), so that metrics are not inflated by near-duplicate windows.

Per-feature scaling

Amplitude is already provided on a logarithmic scale (decibels). Continuous fields are transformed before windowing so that statistics are computed on the training split only and then applied to validation and test data. The log-amplitude, carrier frequency, and pulse width are standardised to zero mean and unit variance using the training-set mean and standard deviation of each field. Each TOA is first min–max scaled to $[0, 1]$ using training-set bounds; within every window, all rescaled TOA values are shifted by subtracting the TOA of the first pulse so that the window starts at time zero. This removes absolute clock offset while preserving intra-window timing structure.

Windows are grouped into mini-batches for optimisation; batch size and per-epoch shuffling of windows are stated in Section 5.3.

4.3 Transformer Encoder Architecture

The encoder $f_{\boldsymbol{\theta}}$ takes the form of a transformer-encoder backbone [8] with a shared trunk followed by two task-specific branches; the overall layout is shown in Figure 4.2. This structure mirrors the dual-embedding formulation introduced in Section 4.1: the trunk produces a single contextualised representation of the input window, which the branches specialise into emitter-oriented and mode-oriented embeddings.

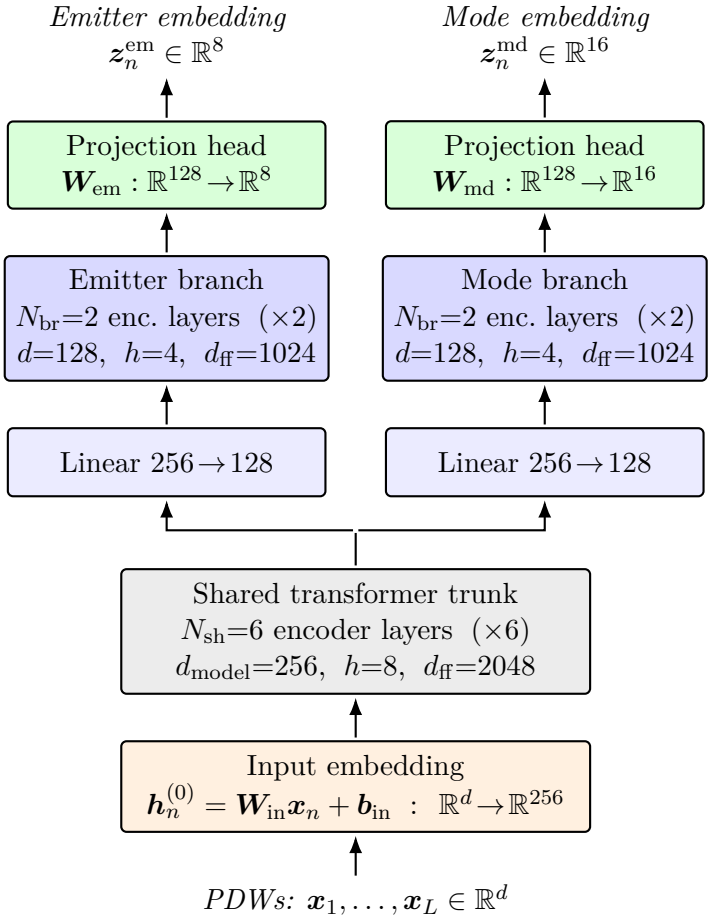

Figure 4.2. Architecture of the encoder $f_{\boldsymbol{\theta}}$. A sequence of PDWs is first lifted to $d_{\mathrm{model}} = 256$ by an input embedding, processed by a shared trunk of 6 transformer-encoder layers, and then split into two parallel branches with their own $256 \to 128$ projection and 2 encoder layers. Branch outputs are mapped by linear projection heads to the emitter embedding space $\mathbb{R}^8$ and the mode embedding space $\mathbb{R}^{16}$.

Input embedding

Each PDW vector $\boldsymbol{x}_n \in \mathbb{R}^d$, normalised as described in Section 4.2, is mapped to a token embedding through an affine projection

$$\boldsymbol{h}_n^{(0)} = \boldsymbol{W}_{\text{in}}\boldsymbol{x}_n + \boldsymbol{b}_{\text{in}}, \qquad \boldsymbol{h}_n^{(0)} \in \mathbb{R}^{d_{\text{model}}}, \tag{4.7}$$

where $\boldsymbol{W}_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times d}$ and d_{model} is the trunk width specified below. No positional encoding is added to $\boldsymbol{h}_n^{(0)}$. The temporal ordering of pulses is already carried by the TOA entry of $\boldsymbol{x}_n$, which makes a separate position signal redundant; consistent with this argument, Gunn et al. [10] report that adding sinusoidal or learnable positional encodings to a transformer deinterleaver yields a small but reproducible drop in performance, attributed to the resulting redundancy with the TOA feature.

Shared trunk

The token sequence $(\boldsymbol{h}_1^{(0)}, \ldots, \boldsymbol{h}_L^{(0)})$ is processed by a stack of N_{sh} standard transformer-encoder layers, each comprising a MHA sub-layer and a position-wise FFN sub-layer with residual connections and layer normalisation around both. Self-attention is unmasked, so every pulse can attend to every other pulse in the window. The trunk operates with hidden dimension $d_{\text{model}} = 256$, $h_{\text{sh}} = 8$ attention heads (so that each head has dimension 32), and an inner FFN dimension of $d_{\text{ff,sh}} = 2048$; $N_{\text{sh}} = 6$ layers are used. Its output is a contextualised token sequence $\boldsymbol{H}^{\text{sh}} = (\boldsymbol{h}_1^{\text{sh}}, \ldots, \boldsymbol{h}_L^{\text{sh}})$ with $\boldsymbol{h}_n^{\text{sh}} \in \mathbb{R}^{256}$, in which information has been mixed across the full window.

Task-specific branches

The trunk output feeds two parallel branches that do not share parameters, producing the dual embeddings of Equation (4.6). Each branch begins with an affine projection that reduces the hidden width from $d_{\text{model}} = 256$ to $d_{\text{br}} = 128$, after which $N_{\text{br}} = 2$ transformer-encoder layers of the same form as the trunk are applied. Within a branch, MHA uses $h_{\text{br}} = 4$ heads (again with per-head dimension 32) and the FFN inner dimension is $d_{\text{ff,br}} = 1024$. The reduced width and depth reflect a deliberate asymmetry: the trunk is intended to carry the heavy lifting of contextual mixing, while the branches only need to specialise the representation to either emitter identity or operating mode. Write the resulting token sequences as $\boldsymbol{H}^{\text{em}} = (\boldsymbol{h}_1^{\text{em}}, \ldots, \boldsymbol{h}_L^{\text{em}})$ and $\boldsymbol{H}^{\text{md}} = (\boldsymbol{h}_1^{\text{md}}, \ldots, \boldsymbol{h}_L^{\text{md}})$ with $\boldsymbol{h}_n^{\text{em}}, \boldsymbol{h}_n^{\text{md}} \in \mathbb{R}^{128}$.

Projection heads

Each branch terminates in a linear projection head that maps every token to its respective embedding space,

$$\boldsymbol{z}_n^{\mathrm{em}} = \boldsymbol{W}_{\mathrm{em}}\boldsymbol{h}_n^{\mathrm{em}} + \boldsymbol{b}_{\mathrm{em}}, \qquad \boldsymbol{z}_n^{\mathrm{md}} = \boldsymbol{W}_{\mathrm{md}}\boldsymbol{h}_n^{\mathrm{md}} + \boldsymbol{b}_{\mathrm{md}}, \tag{4.8}$$

with $\boldsymbol{z}_n^{\mathrm{em}} \in \mathbb{R}^p$ and $\boldsymbol{z}_n^{\mathrm{md}} \in \mathbb{R}^q$ for $p = 8$ and $q = 16$. The mode space is assigned the higher dimensionality because, conditional on emitter identity, mode structure is expected to require a richer geometry to separate subtle waveform regimes, whereas emitter membership induces a coarser partition over the same pulses and is well served by a more compact embedding.

Per-window clustering with HDBSCAN

The projection heads in Equation (4.8) map each pulse to a continuous vector, whereas the latent emitter and mode assignments in Section 4.1 are discrete. HDBSCAN [11, 12] is applied independently to the two embedding trajectories within every analysis window: the L points $\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^{L}$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}_{n=1}^{L}$ each form a separate clustering instance on the corresponding branch. Each window is thus a self-contained pulse train segment in embedding space, consistent with the windowing of Section 4.2, and the effective cardinalities of the inferred emitter and mode partitions follow the density hierarchy instead of fixing K_{em} or K_{md} a priori.

As defined in the framework of Campello et al. [11], HDBSCAN replaces the single global density threshold of DBSCAN [13] with a hierarchy of density estimates: nested level sets are summarised in a condensed tree, and a flat clustering is read off by favouring groups that persist as coherent peaks rather than artefacts of a particular threshold. The same construction supports an explicit noise category for points that are not assigned to any cluster leaf retained after the hierarchy is flattened [11]. The computations use the reference library described by McInnes et al. [12].

The collection of all linear maps in Equations (4.7) and (4.8) together with the attention and FFN weights of the trunk and branches constitutes the parameter vector $\boldsymbol{\theta}$ that the loss in Section 4.4 optimises.

4.4 Loss Function and Clustering Objective

The trainable objective combines contrastive terms on the emitter and mode branches so that both embedding geometries in Equation (4.6) become informative for the unsupervised clusterings described in Section 4.1. This section summarizes the representation-learning rationale, states a generic normalized temperature-scaled cross-entropy (NT-Xent) building block [4], and explains how it is instantiated separately for emitter-centric deinterleaving and for mode-centric structure.

Contrastive learning viewpoint

Contrastive objectives encourage invariance to nuisance transformations while preserving factors that vary across instances [4]. Given two stochastic *views* of the same underlying input (for example two augmentations of the same PDW window), the model should assign similar embeddings to both views, whereas embeddings of unrelated inputs should disagree. Minimizing the resulting classification-style loss is closely related to maximizing a lower bound on mutual information between representations and predictive targets in contrastive predictive coding [5], and more pragmatically it acts as a metric-learning surrogate that spreads the batch on the hypersphere defined by ℓ_2 normalization. A large set of negatives drawn from the mini-batch is essential: without repulsive pressure on unrelated pairs, encoders can trivially collapse to a constant map and still achieve low training error on a naïve agreement loss [4].

Swapping cluster assignments instead of raw embeddings is an alternative that couples representation learning with discrete structure [7]; the present work stays with embedding-space contrast for clarity of exposition, but the same dual-embedding architecture could be paired with a SwAV-style term if prototype banks were introduced.

NT-Xent building block

Let $\phi(\boldsymbol{z}) = \boldsymbol{z}/\|\boldsymbol{z}\|_2$ denote ℓ_2 normalization and let $\tau > 0$ denote the contrastive temperature. Write the scaled cosine similarity as

$$s_\tau(\boldsymbol{a}, \boldsymbol{b}) = \frac{1}{\tau}\, \phi(\boldsymbol{a})^\top \phi(\boldsymbol{b}). \tag{4.9}$$

The temperature τ rescales inner products after normalization: smaller τ sharpens the softmax in Equation (4.10), so the loss concentrates on the hardest negatives and gradients emphasize fine separation on the sphere, whereas larger τ yields a softer distribution closer to uniform and can stabilize optimization when embeddings are poorly separated early in training [4]. All contrastive terms in this thesis share a fixed $\tau = 0.2$, in line with values often used for NT-Xent on normalized features [4].

For a finite index set $\mathcal{I}$ with embeddings $\{\boldsymbol{z}_i\}_{i\in\mathcal{I}}$ and a distinguished *positive* index $p(i) \neq i$ for each anchor $i \in \mathcal{I}$, define the InfoNCE / NT-Xent contribution

$$\mathcal{L}_{\mathrm{NT}}(\mathcal{I}, \{p(i)\}_{i\in\mathcal{I}}) = -\frac{1}{|\mathcal{I}|} \sum_{i\in\mathcal{I}} \log \frac{\exp\Big(s_\tau(\boldsymbol{z}_i, \boldsymbol{z}_{p(i)})\Big)}{\sum_{\substack{j\in\mathcal{I} \\ j\neq i}} \exp\Big(s_\tau(\boldsymbol{z}_i, \boldsymbol{z}_j)\Big)}. \tag{4.10}$$

The denominator collects every candidate in the mini-batch except the anchor itself, so unrelated pairs act as negatives and collapse is discouraged [4]. The same expression is reused below with different choices of index sets, corresponding to different geometric constraints on the emitter and mode spaces.

Emitter-oriented term (deinterleaving)

The emitter branch targets a representation in which pulses from the same physical emitter are close and pulses from different emitters are separated, which is the geometric content expected of a deinterleaving embedding [10]. For each window in a mini-batch, two views are formed by independent draws of the augmentation pipeline (once augmentations beyond deterministic scaling are fixed in Section 4.2); the two views share semantics at the pulse level up to controlled perturbations, so they play the role of a positive pair in the sense of SimCLR [4].

Because emitter identity is a *window-level* hypothesis in dense traffic, the contrastive signal is applied to a per-window summary of the emitter tokens rather than to isolated pulses without batch context. Let $b = 1, \dots, B$ index windows in a batch and let $\bar{\boldsymbol{z}}_b^{\mathrm{em}} \in \mathbb{R}^p$ denote a fixed aggregation of $\{\boldsymbol{z}_{b,n}^{\mathrm{em}}\}_{n=1}^{L}$, such as the arithmetic mean over pulse positions after Equation (4.8). For each b, collect the two pooled view embeddings $\bar{\boldsymbol{z}}_{b,(1)}^{\mathrm{em}}$ and $\bar{\boldsymbol{z}}_{b,(2)}^{\mathrm{em}}$ and form the index set $\mathcal{I}_b^{\mathrm{em}} = \{(b,1),(b,2)\}$ together with the positive pairing that swaps the two views. Concatenate negatives by including the first-view pooled embeddings of all *other* windows in the batch, following the standard "one positive, many negatives" recipe [4]. The emitter loss is the batch average of per-window NT-Xent values,

$$\mathcal{L}_{\mathrm{em}} = \frac{1}{B} \sum_{b=1}^{B} \mathcal{L}_{\mathrm{NT}}\Big(\mathcal{I}_b^{\mathrm{em}} \cup \mathcal{N}_b^{\mathrm{em}},\, p(\cdot)\Big). \tag{4.11}$$

where $\mathcal{N}_b^{\mathrm{em}}$ denotes the chosen negative multiset for window b. This mean over windows matches the implementation choice to score each window's contrastive problem separately before aggregating, which keeps the repulsive set size stable when B changes.

Mode-oriented term

Operating modes induce finer structure within an emitter, often visible only when comparing pulse patterns across longer horizons than a single position index. The mode branch therefore uses the same NT-Xent functional form but mixes *intra-window* comparisons with *inter-window* comparisons so gradients flow both along the token sequence and between windows drawn from the same batch.

Intra-window mode contrast follows Equation (4.10) with $\mathcal{I}$ equal to pulse indices inside a window, positives given by the second augmented view at the same token index, and negatives taken from other positions in the same window and view, which encourages local discrimination of mode-relevant features.

Inter-window mode contrast forms ordered pairs (b, b') of distinct batch indices (for example random pairs or pairs chosen to exploit the half-window stride in Section 4.2 when two windows overlap in time). For each pair, embeddings $\boldsymbol{z}_{b,n}^{\mathrm{md}}$ and $\boldsymbol{z}_{b',n'}^{\mathrm{md}}$ are compared for a small set of index pairs (n, n') (including the diagonal $n = n'$ if both windows are aligned to the same clock after preprocessing). The resulting NT-Xent terms are averaged over sampled pairs and, together with the intra-window term, define

$\mathcal{L}_{\text{md}}$. The construction parallels visual pipelines in which two "crops" of the same image are positives [4], except that the natural pairing is induced by overlapping windows and augmentation rather than by spatial cropping.

Combined objective and use of the projection heads

The overall training loss is the weighted sum

$$\mathcal{L} = \mathcal{L}_{\text{em}} + \lambda_{\text{md}} \mathcal{L}_{\text{md}}, \qquad \lambda_{\text{md}} > 0, \tag{4.12}$$

with λ_{md} treated as a hyperparameter and reported in Chapter 5. An auxiliary DEC-style KL term between soft cluster assignments and a target distribution [6] is a natural extension when trainable centroids or prototypes are added; the experiments chapter states whether such a term is active.

Following common practice in self-supervised encoders [4], the mode embeddings retained for visualization or nearest-neighbor mode analysis are the vectors $\boldsymbol{z}_n^{\text{md}}$ from Equation (4.8), that is, the outputs of the mode linear head. For emitter-centric deinterleaving, the downstream deliverable is primarily the discrete per-pulse assignment $\hat{e}_n$ obtained by clustering pooled emitter embeddings or by reading off a discrete head if one is added later; once those labels are fixed, storing every $\boldsymbol{z}_n^{\text{em}}$ is optional because the continuous representation is not required for the same operational tasks that consume hard emitter tracks.

4.5 Training Procedure

The parameter vector $\boldsymbol{\theta}$ attached to the maps in Equations (4.7) and (4.8) is estimated by minimising the training objective $\mathcal{L}$ in Equation (4.12) on mini-batches of windows. Gradients are obtained by reverse-mode automatic differentiation and used in an adaptive first-order optimiser of the Adam family [14]. Unless Section 5.3 specifies otherwise, the implementation follows the default moment decay rates $(\beta_1, \beta_2) = (0.9, 0.999)$, stabiliser $\epsilon = 10^{-8}$, and bias-corrected running estimates of the first and second moments described by Kingma and Ba [14], as exposed by the PyTorch optimiser interface.

The learning rate follows a cosine decay from a peak value down to a small floor over the course of training, which keeps step sizes comparatively large while the loss landscape is coarse and shrinks them during the final refinement phase. Peak learning rate, weight decay, batch size, number of passes over the training windows, and the exact step counting convention (per epoch versus global step) are hyperparameters and are listed in Table 5.2. At the start of each epoch, training windows are shuffled so that consecutive mini-batches are not ordered by scenario; any fixed random seeds, data-loader worker counts, and reproducibility settings used in the reported runs are stated in Sections 5.1 and 5.3.

Dropout inside the transformer blocks of Section 4.3 follows the usual pattern of stochastic attention and feed-forward masks; dropout rates belong to the same hyperparameter table. Training can be stopped early when a validation score computed on held-out windows (for example the smoothed mini-batch loss or an auxiliary clustering criterion) fails to improve for a fixed patience measured in epochs, in which case the best checkpoint prior to stagnation is retained; patience, validation metric, and checkpoint policy are also recorded in Table 5.2.

If a DEC-style Kullback–Leibler term is added as suggested in Section 4.4, target distributions are recomputed from the student assignments on a schedule analogous to Xie et al. [6], and any sharpening temperature or update period is treated as part of the optimisation protocol reported in Section 5.3. Likewise, the mode-branch weight λ_{md} may be held small during an initial warm-up segment so that the trunk learns coarse temporal structure before the finer mode contrastive gradients dominate; whether such a ramp is used, and its length, is part of the same configuration table. Multi-phase schedules, such as representation pretraining followed by clustering fine-tuning, are compatible with the same tooling; if they are employed, the phases and their objectives are stated in Section 5.3.

4.6 Evaluation Metrics

Let y_n and $\hat{y}_n$ denote reference and predicted cluster indices for pulse $n \in \{1, \dots, N\}$, as in Equations (4.2) and (4.6), instantiated by $(e_n, \hat{e}_n)$ or $(m_n, \hat{m}_n)$. The contingency is $n_{ij} = \sum_{n=1}^{N} \mathbf{1}[y_n = i,\, \hat{y}_n = j]$ with marginals $a_i = \sum_j n_{ij}$ and $b_j = \sum_i n_{ij}$.

Adjusted Rand index

The ARI [15] is

$$\mathrm{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \big[\sum_i \binom{a_i}{2}\big]\big[\sum_j \binom{b_j}{2}\big] / \binom{N}{2}}{\frac{1}{2}\Big[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\Big] - \big[\sum_i \binom{a_i}{2}\big]\big[\sum_j \binom{b_j}{2}\big] / \binom{N}{2}}, \qquad \mathrm{ARI} \le 1\,. \tag{4.13}$$

Perfect agreement up to relabelling gives $\mathrm{ARI} = 1$.

Entropies, mutual information, and normalizations

Shannon entropies of the marginals are

$$H(Y) = -\sum_{i:\, a_i > 0} \frac{a_i}{N} \log \frac{a_i}{N}\,, \qquad H(\hat{Y}) = -\sum_{j:\, b_j > 0} \frac{b_j}{N} \log \frac{b_j}{N}\,. \tag{4.14}$$

The empirical mutual information is [16]

$$\mathrm{MI}(Y;\hat{Y}) = \sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{N\, n_{ij}}{a_i b_j}\,, \tag{4.15}$$

with $0 \log 0 = 0$. A NMI score is obtained by dividing MI by a function of $\{H(Y), H(\hat{Y})\}$; we use the arithmetic-mean denominator [16],

$$\mathrm{NMI}(Y;\hat{Y}) = \frac{2\,\mathrm{MI}(Y;\hat{Y})}{H(Y) + H(\hat{Y})}\,, \qquad H(Y) + H(\hat{Y}) > 0\,, \tag{4.16}$$

and set $\mathrm{NMI} = 0$ if $H(Y) + H(\hat{Y}) = 0$. Let $\mathbb{E}[\mathrm{MI}]$ be the expectation of MI under the permutation null that fixes one partition while permuting labels subject to fixed cluster sizes [16]. The AMI is

$$\mathrm{AMI}(Y;\hat{Y}) = \frac{\mathrm{MI}(Y;\hat{Y}) - \mathbb{E}[\mathrm{MI}]}{\frac{1}{2}\Big(H(Y) + H(\hat{Y})\Big) - \mathbb{E}[\mathrm{MI}]}\,, \tag{4.17}$$

when the denominator is positive; otherwise $\mathrm{AMI} \equiv 0$.

Clustering accuracy

Cluster indices are arbitrary up to a bijection ϕ on label symbols. The ACC is the largest agreement after an optimal relabeling,

$$\mathrm{ACC} = \frac{1}{N} \max_{\phi \in \Phi} \sum_{n=1}^{N} \mathbf{1}\Big[y_n = \phi(\hat{y}_n)\Big]\,, \tag{4.18}$$

where Φ is the set of bijections between the active reference labels and the active predicted labels when their counts coincide; if cardinalities differ, the same maximum is computed by the Hungarian (linear-sum assignment) algorithm on the matrix (n_{ij}) with cost $-n_{ij}$ [17].

V-measure

With conditional entropies

$$H(Y \mid \hat{Y}) = -\sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{b_j}\,, \qquad H(\hat{Y} \mid Y) = -\sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{a_i}\,, \tag{4.19}$$

homogeneity and completeness are [18]

$$h = 1 - \frac{H(Y \mid \hat{Y})}{H(Y)}\,, \qquad c = 1 - \frac{H(\hat{Y} \mid Y)}{H(\hat{Y})}\,, \tag{4.20}$$

with $h = 1$ if $H(Y) = 0$ and $c = 1$ if $H(\hat{Y}) = 0$. The V-measure is

$$\mathrm{V}(Y, \hat{Y}) = \frac{2hc}{h + c}\,, \qquad h + c > 0\,, \tag{4.21}$$

and $\mathrm{V} = 0$ if $h = c = 0$.

Figures

Two-dimensional projections of $\{\boldsymbol{z}_n^{\mathrm{em}}\}$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}$ coloured by (e_n, m_n) complement Equations (4.13), (4.16) to (4.18) and (4.21); protocols and numbers appear in Chapter 5.

4.7 Baseline Methods

The baselines isolate the contribution of the dual-branch layout in Section 4.3 and of the mode-oriented term $\mathcal{L}_{\mathrm{md}}$ in Equation (4.12) relative to a single-stack encoder and to classical clustering without a sequence model.

The first neural baseline removes the emitter and mode branches and replaces them with one stack of eight transformer-encoder layers of the same functional form as the shared trunk (hidden width, head count, and FFN expansion match that trunk so the comparison stresses topology and objective rather than a change in per-layer width). Token embeddings $\boldsymbol{h}_n^{(0)}$ from Equation (4.7) pass through this stack; a single linear head then maps each contextualised token to $\boldsymbol{z}_n \in \mathbb{R}^p$, using the same output dimension p as the emitter branch in the full model. Training minimises only the emitter-oriented contrastive loss $\mathcal{L}_{\mathrm{em}}$ in Equation (4.11); $\mathcal{L}_{\mathrm{md}}$ and its inter-window structure are omitted, so no auxiliary geometry is trained for operating modes. This configuration tests whether one representation, optimised purely for window-level deinterleaving, suffices for both emitter- and mode-level clusterings reported in Chapter 5.

The second baseline is HDBSCAN [12], a hierarchical density-based method that does not estimate a transformer. It clusters fixed-length vectors built deterministically from the per-feature scaled PDW windows in Section 4.2, for example by per-channel summary statistics or a flattened layout of pulses and features; the exact vectorisation is fixed in Chapter 5. HDBSCAN separates gains due to learned sequence context from what density-based Euclidean clustering already recovers on hand-specified summaries.

Further comparisons—k-means on raw window vectors or on autoencoder embeddings, DEC-style objectives when centroids are used, SwAV-style assignment prediction [7], and a supervised classifier upper bound when labels are available—are listed with their configurations in Chapter 5.

CHAPTER 5

Experiments and Results

5.1 Experimental Setup

All neural models in this chapter were trained and evaluated on a single workstation with one NVIDIA A100 GPU (80 GB high-bandwidth memory; some system utilities report a capacity closer to 82 GB because of binary versus decimal gigabyte conventions). The implementation uses Python with PyTorch; exact package versions are pinned in the environment file shipped with the thesis code repository when that repository is published.

Random seeds control weight initialization, training-window shuffling each epoch, and stochastic augmentations. Unless an experiment explicitly varies the seed, the same seed set is reused so that ablations in Section 5.5 are comparable under identical noise conditions. Concrete seed integers, per-run overrides, and mean $\pm$ standard deviation over multiple seeds for quantitative scores are reported with the tables in Section 5.4.

5.2 Datasets

All experiments in this chapter use synthetic PDW streams generated by the proprietary scenario simulator introduced in Section 4.2. The simulator supplies time-ordered pulse records together with ground-truth emitter identity and operating mode for each pulse; those labels are withheld from the training objective and are used only to compute the clustering metrics in Sections 4.6 and 5.4. Operational ELINT recordings are not included, which sidesteps distribution shift and disclosure constraints while still exercising mixture traffic, mode changes within emitters, and variable train density.

Each *scenario* is one simulated timeline: a sequence of PDWs whose length and the number of concurrent emitters vary across draws so that the encoder sees both sparse and crowded electromagnetic pictures. Scenarios are disjointly assigned to training, validation, and test partitions before any windowing; all normalisation statistics in Section 4.2 are fit on the training partition only and applied unchanged to validation and test pulses. Training windows use stride $L/2$ with $L = 1024$ pulses, whereas validation and test windows use stride L, so the effective number of windows per scenario differs by split even when raw pulse counts are similar.

Table 5.1 summarises corpus-level figures. Emitter and mode columns count distinct simulator labels that appear anywhere in the corresponding split; they are upper bounds on the cardinality that unsupervised clustering must recover, because not every label may be present in every window. The numeric entries are placeholders until the final

Table 5.1. Synthetic scenario corpus used for training and evaluation. Dashes mark quantities still being reconciled with the export manifest.

Split	Scenarios	Pulses (approx.)	Distinct emitters	Distinct modes
Train	—	—	—	—
Validation	—	—	—	—
Test	—	—	—	—
All	—	—	—	—

Table 5.2. Primary optimisation hyperparameters for the proposed model.

Setting	Value
Optimiser	Adam [14] with PyTorch defaults $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\epsilon = 10^{-8}$
Peak learning rate $\eta_{\max}$	—
Minimum learning rate $\eta_{\min}$	0 (cosine floor; PyTorch default)
Cosine period $T_{\max}$	30 scheduler steps, one step per training epoch
Maximum epochs	30
Early stopping patience	7 epochs without improvement on the validation loss
Early stopping checkpoint	Weights from the epoch with lowest validation loss so far
Dropout (attention and feed-forward)	0.1 throughout the transformer blocks
Batch size B (windows per step)	—
Mode loss weight λ_{md}	—
Weight decay	—

export from the simulator pipeline is frozen; the intended reporting convention is scenarios for volume, pulses for aggregate exposure, and distinct labels for emitter and mode coverage.

5.3 Hyperparameters and Training Details

This section collects numerical training choices deferred from Sections 4.3 to 4.5. Unless Section 5.5 states otherwise, baseline models use the same epoch budget, hardware (Section 5.1), and early-stopping protocol so that differences reflect architecture and objective rather than compute.

The contrastive temperature is fixed at $\tau = 0.2$ as in Section 4.4. Table 5.2 summarises the optimisation schedule and regularisation for the proposed dual-branch encoder; entries marked with an em dash are still being matched to the released training script and will be filled before the final thesis submission.

The cosine scheduler follows the PyTorch `CosineAnnealingLR` convention: after each epoch the learning rate is updated from $\eta_{\max}$ toward $\eta_{\min}$ over $T_{\max} = 30$ steps, which aligns the period with the nominal training horizon. If early stopping triggers first, training halts at the best validation checkpoint and the final few scheduler steps may not be executed.

5.4 Quantitative Results

Placeholder paragraph.

5.5 Ablation Studies

Placeholder paragraph.

5.6 Qualitative Analysis

Placeholder paragraph.

CHAPTER 6

Discussion

6.1 Interpretation of Results

Placeholder paragraph.

6.2 Comparison with Related Work

Placeholder paragraph.

6.3 Limitations

Placeholder paragraph.

6.4 Implications

Placeholder paragraph.

CHAPTER 7

Conclusion

7.1 Summary

Placeholder paragraph.

7.2 Answers to the Research Questions

Placeholder paragraph.

7.3 Future Work

Placeholder paragraph.

References

[1] R. G. Wiley, *ELINT: The Interception and Analysis of Radar Signals*. Artech House, 2006.

[2] M. I. Skolnik, *Radar Handbook*, 3rd ed. McGraw-Hill, 2008.

[3] Y. LeCun. "A path towards autonomous machine intelligence". version 0.9.2, OpenReview, Accessed: May 13, 2026. [Online]. Available: `https://openreview.net/forum?id=BZ5a1r-kVsf`

[4] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations", in *International Conference on Machine Learning (ICML)*, 2020.

[5] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, arXiv preprint arXiv:1807.03748, 2018. arXiv: `1807.03748 [cs.LG]`.

[6] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis", in *International Conference on Machine Learning (ICML)*, 2016, pp. 478–487.

[7] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, "Unsupervised learning of visual features by contrasting cluster assignments", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding", in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[10] E. Gunn, A. Hosford, D. Mannion, J. Williams, V. Chhabra, and V. Nockles, *Radar pulse deinterleaving with transformer based deep metric learning*, arXiv preprint arXiv:2503.13476, 2025. arXiv: `2503.13476 [cs.LG]`.

[11] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, "Hierarchical density estimates for data clustering, visualization, and outlier detection", *ACM Transactions on Knowledge Discovery from Data*, vol. 10, no. 1, 2015. DOI: `10.1145/2733381`

[12] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering", *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017. DOI: `10.21105/joss.00205`

[13] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise", in *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[14] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, arXiv preprint arXiv:1412.6980, 2017. arXiv: 1412.6980 [cs.LG].

[15] L. Hubert and P. Arabie, "Comparing partitions", *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[16] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance", *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[17] H. W. Kuhn, "The Hungarian method for the assignment problem", *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955. DOI: 10.1002/nav.3800020109

[18] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure", in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 2007, pp. 410–420.

APPENDIX A

Additional Results

Placeholder paragraph.

APPENDIX B

Implementation Details

Placeholder paragraph.

APPENDIX C

Notation Reference

Placeholder paragraph.